FUNDIGRID · ENGINEERING HANDOVER

Backend Migration & Deployment Summary

Node/Express → ASP.NET Core 10, SignalR realtime, and the hosting / database decision for taking FundiGrid to real users.

ASP.NET Core 10 | EF Core + Npgsql | SignalR | Docker

Date	12 November 2026
Branch	`feat/aspnet-backend`
Commits	`4461622` migration · `9fb3151` pool-size fix
Node backup	Git tag `node-backend-final` (pushed)
Status	Backend port complete and validated; database must be restored or replaced before deployment

Contents

1 Executive summary

2 What changed

3 Validation performed

4 Render deployment guide

5 Database options

6 Production hosting & database

7 Pre-launch checklist

8 Environment variable reference

9 Next steps

1. Executive summary

The FundiGrid backend has been migrated from Node/Express to **ASP.NET Core 10**. The public REST contract, JWT sessions, bcrypt password hashes, Postgres schema, and deployment hostname are preserved. Socket.IO has been replaced with a **SignalR** hub using the same method and event names, and both the web and mobile clients have been updated.

50
API endpoints (49 controller routes + health)

32
end-to-end checks passing

15
Postgres tables mapped with EF Core

100%
SignalR location round-trip verified

> **Current blocker.** The Supabase project `fqblwkktbucxngyxylnn` is unreachable — the pooler returns `tenant/user not found` and the REST host does not resolve. The Node backend fails identically, so this is a database availability problem, not a migration problem. Restore the project (to keep data) or point `DATABASE_URL` at a new Postgres (Neon, Render, Azure) before deploying.

Recommendation at a glance

Decision	Recommendation
Database now	Restore the existing Supabase project if the data matters; otherwise use **Neon**.
First production stack	**Render Starter + Neon paid** (Frankfurt / `eu-central-1`), Vercel for web, EAS for mobile.
When revenue/latency justifies	Move API + DB to **Azure South Africa North** for ~50–90 ms latency to Uganda.
Do before real users	Move base64 images out of Postgres; integrate real MTN/Airtel payments; add SMS rate limits and backups.

2. What changed

Area	Before (Node/Express)	After (ASP.NET Core 10)
Runtime	Node 22, Express 4	.NET 10 LTS, ASP.NET Core controllers
Data access	`@supabase/supabase-js` over PostgREST + a Mongoose-like shim	EF Core 10 + Npgsql, direct Postgres connection
Schema	`sql/schema.sql`, applied by `npm run migrate`	Same `sql/schema.sql`, applied by `dotnet run -- --migrate`
Auth	JWT `{ id, role }`, 30 days, bcryptjs	Identical JWT claims/expiry, BCrypt.Net-Next (existing hashes verify)
Realtime	Socket.IO, 12 events	SignalR hub at `/hubs/realtime`, same event names
OTP / SMS	EgoSMS over Node `https`	EgoSMS over `HttpClient`, same dev-mode rules
Google sign-in	`google-auth-library`	`Google.Apis.Auth`
Supabase SDK	URL + service-role key on every query	Removed — only `DATABASE_URL` is used
Deployment	Node buildpack on Render	Multi-stage Docker image, same Render service hostname

Contract cleanup applied

- JSON is **camelCase everywhere** — `createdAt`, `updatedAt`, `isOnline`.
- `id` **only**; the old `_id` fallbacks were removed from mobile.
- Optional fields are always present, with `null` when absent.
- Removed unused placeholders: `GET /api/chat`, `GET /api/ai`, `GET /api/maps/route`, and the admin root.
- Routes, `{ success, code?, message }` error envelopes, and enum values are unchanged.

Repository artifacts

Item	Location
Migration commit	`4461622` — 112 files, +6,406 / −6,847
Connection-pool fix	`9fb3151` — Npgsql pool capped at 20 for Supabase limits
Branch / PR	`feat/aspnet-backend` · github.com/fundigrid/fundigrid
Node rollback	Tag `node-backend-final` (local and GitHub)
New backend files	`backend/Program.cs`, `Controllers/`, `Data/`, `Entities/`, `Services/`, `Hubs/`, `Dockerfile`

3. Validation performed

All validation ran against a temporary local Postgres 17 cluster on port `55432` (the Supabase project was already unreachable). The cluster was stopped after testing.

REST end-to-end (32 checks)

Flow	Verified behaviour
Admin & accounts	Admin login, create fundi/client, timestamps, approve verification, role guard (client → admin = 403)
Auth	Phone login, OTP send/verify (dev mode), profile, platform settings
Directory	Verified-only fundi listing, category/location filters
Booking lifecycle	Create → accept → quote → client accepts → ON_THE_WAY → ARRIVED → IN_PROGRESS → fundi finished → client confirms
Money	Payment, idempotent re-pay, receipt, fundi wallet credit, transaction ledger
Reviews	Submit review, rating recomputation
Admin	Stats/revenue, bookings, requests with client names, audit trail, settings update
Progress	Timeline steps all marked complete

Realtime and builds

Check	Result
SignalR `update_location` → `fundi_location`	**PASS** — client received the booking id and coordinates
SignalR JWT handshake	**PASS** — token via `access_token` query
`dotnet build`	**PASS** — 0 warnings, 0 errors
`dotnet publish`	**PASS** — `sql/schema.sql` included in output
Mobile `tsc --noEmit`	**PASS**
Web `vite build`	**PASS**
Schema migration	**PASS** — applied cleanly and idempotently

> **Not yet tested against production data.** Because the Supabase project is unavailable, the port was validated on an empty database. Once the real database is restored, run the migration and a read-only smoke test (login, fundi list, bookings, wallet balances) before switching traffic.

4. Render deployment guide

Service settings

Setting	Value
Runtime / Language	**Docker**
Branch	`main` after merge (or `feat/aspnet-backend` to test)
Root Directory	`backend`
Dockerfile Path	`Dockerfile`
Docker Build Context	`backend`
Health Check Path	`/api/health`
Instance Type	Starter or higher (Free sleeps after 15 minutes)
Auto-Deploy	On
`PORT`	**Do not set** — Render injects it; `Program.cs` binds to it

> **Keeping the URL.** The `*.onrender.com` hostname follows the service name. If Render will not let you switch the existing service to Docker, create a new Docker service with a temporary name, verify it, then delete the old service and rename the new one to `fundigrid`. Web and mobile need no URL changes when the hostname stays `fundigrid.onrender.com`.

First deploy

1. Set the environment variables in section 8, including `RUN_MIGRATIONS=true`.
2. Deploy and watch the logs for:

```
Schema applied from .../sql/schema.sql
Admin user ready: ...
Now listening on: http://0.0.0.0:<port>
```

3. Open `https://fundigrid.onrender.com/api/health` — expect `{"status":"ok",...}`.
4. Sign in on the web admin with `ADMIN_EMAIL` / `ADMIN_PASSWORD`.
5. Set `RUN_MIGRATIONS=false` and redeploy. Prefer Render's pre-deploy command (`dotnet FundiGrid.Api.dll --migrate`) for future schema changes.

WebSockets and migrations

- Render supports WebSockets without extra configuration; SignalR works as-is.
- `RUN_MIGRATIONS=true` runs the idempotent schema on every boot. It is safe but slower and briefly locks tables — use it only for the first deploy or a pre-deploy step.
- The Docker image includes `sql/schema.sql`; migrations do not depend on the repo checkout.

5. Database options

Supabase project is offline. The configured host `aws-1-eu-west-1.pooler.supabase.com` rejects the tenant, and the REST host does not resolve. Restore the project in the Supabase dashboard if the data matters; if it was deleted, the data is gone and a new database starts empty.

Option	Free tier	Paid	Caveats
Supabase	500 MB DB, no expiry; project pauses after ~7 days inactivity and needs a manual restore	Pro ~$25/mo, 8 GB DB, 7-day PITR, 100 GB storage	Best if you keep your existing data or want Storage for images
Neon	100 CU-hours/project/month, ~0.5–1 GB, autosuspend after ~5 min, **auto-resumes**, no expiry	Usage-based, ~$0.106/CU-hour + storage, no minimum	Best DB-only free tier; no Africa region
Render Postgres	1 GB, **expires 30 days after creation** (14-day grace, then deleted)	From ~$6/mo (256 MB), $19/mo (1 GB)	Free is a trial only — do not store real user data
Azure PG Flexible	None	~$15–30/mo, automated backups, South Africa North	Best latency/compliance for Uganda when paired with Azure

Supabase vs Neon for FundiGrid

Criterion	Supabase	Neon
Existing data	**Yes** — project already holds users/bookings	Starts empty
Idle behaviour	Pauses; manual restore required	**Auto-suspends and auto-resumes**
Expiry	None	None
Storage	500 MB + 1 GB object storage	~0.5–1 GB
Extras	Auth, Storage, Realtime (currently unused)	Branching, DB-only
Fit for this backend	Good if you restore the data or adopt Storage	**Better as a pure Postgres host**

Verdict. Restore Supabase if the data matters — that is the only option that keeps existing users, bookings, wallets, and reviews. If the project is gone or the data is disposable, choose **Neon**: it wakes itself up, never expires, and matches a DB-only ASP.NET backend.

6. Production hosting & database

Hosting platforms

Platform	.NET + SignalR fit	Cost (small prod)	Notes
Render	Good — Docker, WebSockets, health checks	~$7/mo web	Easiest migration from current setup; use Starter, not Free
Azure App Service	**Best** — native .NET, slots, App Insights	~$13–25/mo	Best long-term; Azure SignalR Service for scale-out
Fly.io	Good — Docker, WebSockets, **Johannesburg**	~$5–15/mo	Lowest latency to Uganda if the DB is also in `jnb`
Railway	Good — Docker, WebSockets	~$5–20/mo	Very fast setup, usage-based
DigitalOcean App Platform	Good — Docker	~$12/mo + DB	Predictable pricing
AWS ECS/ Fargate	Works, more ops	~$20–40/mo	Only if you already know AWS
Vercel	**No** for the API	—	Keep for the React web app only; serverless does not suit SignalR

Three concrete stacks

Stack	Components	Cost	When
1. Budget launch	Render Starter (Frankfurt) + Neon paid (`eu-central-1`) + Vercel + EAS + Cloudflare R2 for images	~$15–25/mo	**Recommended now**
2. .NET-native	Azure App Service + Azure PG Flexible (**South Africa North**) + Blob Storage + Application Insights	~$40–70/mo	When reliability and support matter
3. Lowest latency	Fly.io `jnb` or Azure SA North + Azure PG SA North / AWS RDS `af-south-1`	~$30–60/mo	When Ugandan users report lag

Region guidance

User location	Recommended region	Typical latency
Uganda / East Africa	Johannesburg / Cape Town (Azure SA North, AWS `af-south-1`, Fly `jnb`)	~50–90 ms

Uganda (budget option)	Frankfurt / EU (`eu-central-1`)	~120–160 ms
Global / diaspora	Multi-region or nearest major region	Varies

> Always place the API and the database in the **same region**. Splitting them across continents makes every query pay the round-trip and is the most common cause of a "slow" backend after launch.

7. Pre-launch checklist

#	Item	Why it matters
1	**Move images out of Postgres**	Portfolio and verification images are base64 in the DB (up to 6 × ~1.5–2.5 MB each). A few fundis can fill 500 MB–1 GB. Use Cloudflare R2, S3, or Azure Blob.
2	**Real payments**	`POST /payments/pay` is simulated. Production needs MTN MoMo / Airtel Money collection, webhooks, reconciliation, escrow and refunds.
3	**SMS abuse controls**	Every OTP costs money. Add per-phone/per-IP rate limits, resend cooldowns, and monitoring for SMS bombing.
4	**Backups + PITR**	Neon paid, Supabase Pro, and Azure PG offer point-in-time restore. Render free has none. Test a restore once.
5	**Secrets management**	Keep `JWT_SECRET`, `DATABASE_URL`, and SMS keys in the platform secret store. Rotate the admin password. Never commit `.env`.
6	**Monitoring + alerts**	Sentry or Application Insights; uptime checks on `/api/health`; alerts for 5xx spikes and SMS failures.
7	**SignalR scale-out plan**	One instance is fine at launch. A second instance needs a Redis backplane or Azure SignalR Service, or location updates only reach part of the fleet.
8	**Legal / compliance**	You store phone numbers, IDs, and verification documents. Uganda's Data Protection and Privacy Act applies — encryption, retention, and deletion policy.
9	**Migrations as a release step**	Use Render's pre-deploy command (`dotnet FundiGrid.Api.dll --migrate`) or an Azure release task instead of migrating on every boot.
10	**Domain + TLS**	`api.fundigrid.com` on the API, `fundigrid.com` on Vercel; update `VITE_API_URL` and the mobile `EXPO_PUBLIC_API_URL`.

8. Environment variable reference

Variable	Required	Purpose / notes
`DATABASE_URL`	Yes	Postgres URI. Session pooler (5432) preferred; transaction pooler (6543) is supported. Include `?sslmode=require` where needed.
`JWT_SECRET`	Yes	**Must match the Node value**, or every existing session is invalidated.
`ASPNETCORE_ENVIRONMENT`	Yes (prod)	`Production`. Controls dev-mode safety for OTP.
`RUN_MIGRATIONS`	First deploy	`true` to apply `sql/schema.sql` on boot; set `false` afterwards.
`ADMIN_NAME`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`	Yes	Admin re-seeded on every boot (same behaviour as Node).
`COMMS_ENDPOINT`, `COMMS_USERNAME`, `COMMS_API_KEY`	Yes	EgoSMS credentials.
`COMMS_SENDER_ID`, `COMMS_TIMEOUT_MS`	Optional	Sender name (default `Ufundi`) and gateway timeout (default 15000 ms).
`COMMS_DEV_MODE`	Yes	**`false` in production**. When true, OTP codes are returned in the API response.
`COMMS_LOG_OTP`	Recommended	`true` while verifying SMS delivery; `false` once trusted.
`GOOGLE_WEB_CLIENT_ID`	Yes	Google sign-in audience validation.
`GOOGLE_MAPS_API_KEY`	Optional	Feature flag / maps integration.
`GROQ_API_KEY`, `GROQ_MODEL`	Optional	Unused by the current backend; safe to keep.
`PORT`	Auto	Injected by Render. Do not set manually.
`SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`	**Remove**	No longer read. The service-role key bypasses RLS, so it should not live in an environment that does not use it.

> **Why the Supabase keys are gone.** The Node backend used `@supabase/supabase-js`, an HTTP client for Supabase's PostgREST API, authenticated with the service-role key. The new backend connects directly to Postgres with Npgsql using `DATABASE_URL`; it never calls Supabase's REST, Storage, Auth, or Realtime APIs.

9. Next steps

1. **Restore the Supabase project** (or create a Neon project) and copy the session-pooler connection string.
2. **Apply the schema:**

   ```
   cd backend
   DATABASE_URL="postgresql://...?sslmode=require" dotnet run -- --migrate
   ```

3. **Smoke-test against the real data:** login, fundi directory, bookings, wallet balances, admin stats.
4. **Configure Render** as a Docker service (section 4) and set the env vars (section 8). First deploy with `RUN_MIGRATIONS=true`.
5. **Verify** `/api/health` and the admin login, then set `RUN_MIGRATIONS=false`.
6. **Merge** `feat/aspnet-backend` into `main` and let auto-deploy run.
7. **Release the mobile app** with `@microsoft/signalr` (EAS build) and deploy the updated web app (Vercel).
8. **Plan the production hardening** in section 7 — images, payments, SMS limits, backups, monitoring.
9. **Revisit hosting** when usage grows: Azure South Africa North for latency, Azure SignalR Service for scale-out, object storage for media.

> **Rollback.** If the new backend needs to be reverted during cutover, deploy the `node-backend-final` tag with the original Node buildpack and environment variables. The database schema is shared, so no data conversion is required.

Prepared for the FundiGrid engineering handover · 12 November 2026. Source: branch `feat/aspnet-backend` (commits `4461622`, `9fb3151`).